

Design Pattern Identification and Evolution

A FastAPI-Based Energy Management Application

TEAM B13

AM.SC.U4CSE23306 - Anand Narayan

AM.SC.U4CSE23346 - S H Saimadhav

AM.SC.U4CSE23365 - Vaishnav P Nair

AM.SC.U4CSE23369 - Vishnu M

Parts 1-3 evidence package

7 October 2026

Submission status

This report consolidates the **Part 1 baseline**, **verified Part 2 evolution**, and **completed Part 3 cross-language comparison**. The three code directories are intentionally empty for the team's final source upload.

Area	Status	Evidence available
Part 1	Complete	Seven-area investigation, architecture, pattern inventory, UML source, code excerpts, LLM record
Part 2	Complete	Three evolution scenarios, source mapping, diffs, tests, updated UML, LLM record
Part 3	Complete	Two patterns, four languages, shared cases, results, metrics, timings, and LLM record
Code upload	User action	Part1/code, Part2/code, and Part3/code are intentionally empty

Application description

The repository implements a FastAPI Energy Management System for simulated load scheduling. It manages portfolios and loads, obtains forecasts, creates schedules through MILP with a heuristic fallback, records forward and backward metrics, and streams runtime state through WebSocket.

The case study investigates energy consumption, smart meters, users, tariff management, renewable-energy sources, alerts, and reporting. Repository evidence separates implemented behavior from possible future behavior.

Submission alignment

The team folder is B13. Dataset files use the required columns, store team-relative source paths, and distinguish measured results from team-confirmed results. Missing execution times remain blank rather than estimated.

Baseline architecture

Layer	Responsibilities
Interface	FastAPI REST endpoints, queue-based runtime, WebSocket connections
Application	Portfolio management, metric advancement, planning, plan application
Domain	Portfolio, Load, Metrics, Plan, status and metric types, domain ports
Infrastructure	Simple forecast provider, simple portfolio repository, serialization

Part 1 - Pattern discovery

Investigation results

Area	Status	Verified finding
Energy consumption	Present	Loads consume simulated grid and renewable energy; metrics and solvers track the draw.
Smart meters	Absent	No meter entity, meter-reading API, telemetry port, or physical-device adapter exists.
Users	Absent	No user, account, authentication, role, or authorization model exists.
Tariff management	Partial	GRID_PRICE affects cost optimization, but the baseline has no explicit tariff policy or management API.
Renewable sources	Present	Production, renewable load draw, and spill participate in planning as aggregate values.
Alerts	Absent	WebSocket streams state but no domain rule evaluates an alert condition.
Reporting	Partial	Metrics and streaming provide report inputs; no report generator or export API exists.

Baseline pattern inventory

The repository shows Ports and Adapters through forecast and repository abstractions, Repository for portfolio persistence, FastAPI dependency injection for runtime access, strategy-like selection between MILP and heuristic solvers, and observer-like delivery to WebSocket subscribers.

Part 1 establishes the evolution boundary: pricing exists without explicit tariff behavior, renewable energy exists without named source types, and metrics exist without alert rules.

Part 2 - Software evolution

Scenario 1: dynamic tariffs

Strategy encapsulates price calculation behind `TariffStrategy.get_price(time_index)`. Flat, time-of-use, and dynamic implementations supply interchangeable policies. The forecast provider writes the selected result into the existing `GRID_PRICE` metric.

```
TariffStrategy
|-- FlatTariff
|-- TimeOfUseTariff
`-- DynamicTariff
|
v
GRID_PRICE -> Metrics -> build_params -> Solver
```

The solvers remain independent of tariff selection. One simulation step represents one hour for time-of-use behavior, and a supplied dynamic series repeats for the running demonstration.

Scenario 2: renewable-source integration

RenewableSource strategies make solar and wind behavior explicit. `EnergySourceFactory` creates a source from configuration. The provider records each source's output, sums the values, and writes the aggregate to `REN_PROD`. Existing solver inputs remain unchanged.

Solar uses a repeatable daylight profile. Wind multiplies capacity by supplied availability fractions. Both behaviors are simulations rather than physical measurements.

Scenario 3: consumption alerts

After backward metrics receive a completed step's grid and renewable draw, the application publishes one `ConsumptionEvent`. `ConsumptionSubject` notifies its subscribers. `ConsumptionAlertObserver` records a load when combined consumption is greater than the configured threshold.

```
Completed grid + renewable draw
-> ConsumptionEvent
-> ConsumptionSubject
-> Threshold observer
-> REST and WebSocket output
```

Pattern-to-source mapping

Scenario	Pattern	Principal evolved sources
Dynamic tariffs	Strategy	domain/tariff/*; manage_energy.py; simple_forecast_provider.py; PUT /tariff
Renewable integration	Strategy + Factory	domain/energy_sources/*; energy_configuration.py; source routes; runtime snapshot
Consumption alerts	Observer	domain/alerts/*; ems_service.advance; alert routes; runtime snapshot

Before and after

Concern	Before	After
Grid price	Provider-generated scalar	Strategy-selected scalar stored as GRID_PRICE
Renewable production	One aggregate mock value	Named solar and wind strategies aggregated into REN_PROD
Consumption monitoring	Draw stored in Metrics	Completed-step event evaluated by subscribed observers
Runtime output	Portfolio and metrics	Adds source breakdown and alert records
Solver input	Price and renewable series	Same parameter contract; solver design preserved

The refactoring places variable behavior at established boundaries. Tariffs and sources feed the provider, while consumption events occur after metric advancement. This keeps the optimization core stable and gives every new behavior a separate test surface.

Verification and metrics

Check	Verified result
Full automated suite	13 tests passed in 0.53 seconds
MILP regression smoke test	Model build and solve completed
Tariffs	Flat, time-of-use, dynamic series, provider integration, invalid request
Renewables	Solar and wind behavior, factory creation, aggregation, REST and WebSocket output
Alerts	Threshold boundary, event publication, subscription removal, combined draw, REST and WebSocket output

Compatibility findings

The original solver imports PULP_CBC_CMD, so the evolved requirements constrain PuLP below version 4. A deprecation warning remains, but the MILP smoke test succeeds. The Starlette test client emits a transport deprecation warning without test failure.

Verified environment and measurement rules

Part 2 verification used Windows, Python 3.12.14, FastAPI 0.142.2, Pydantic 2.13.5, pytest 9.1.1, PuLP 3.3.2, and httpx 0.28.1. Part 3 excludes blank lines, comments, and tests from LOC. Available timings use one warmup and five measured process runs, then report the median.

LLM use and human verification

The LLM supported repository classification, pattern selection, source mapping, evidence drafting, and equivalent cross-language implementations. Human verification checked claims, executed the evolved application tests, and compared language results with shared cases. Prompt, input, output, and verification records cover Parts 1, 2, and 3.

Important interpretations

Decision	Reason
WebSocket is monitoring, not an alert system	The baseline broadcasts values but evaluates no business rule.
Renewables are evolved, not introduced from zero	REN_PROD, LOAD_REN_DRAW, and REN_SPILL already affect planning.
Factory is supporting	Strategy and Observer provide the two strongest cross-language problems.
Claims remain simulation-scoped	No physical meter, external notification service, or persistent alert store was tested.

Part 3 - Cross-language analysis

Strategy is implemented through dynamic tariffs and Observer through consumption alerts. Java, Python, JavaScript, and C++ use the same conceptual inputs and expected behavior.

Implementation and verification matrix

Pattern	Language	LOC	CC	Result	Median ms
Strategy	Java	72	5	17/17 user-confirmed	-
Strategy	Python	51	3	17/17 tool-captured	20.552
Strategy	JavaScript	71	5	17/17 tool-captured	41.596
Strategy	C++	82	5	17/17 user-confirmed	-
Observer	Java	86	3	10/10 user-confirmed	-
Observer	Python	43	3	10/10 tool-captured	45.165
Observer	JavaScript	63	3	10/10 tool-captured	42.924
Observer	C++	67	3	10/10 user-confirmed	-

Measurement scope

Python 3.12.14 and Node.js 24.21.0 results were captured in the evidence workspace. Each timing includes process startup, the language runner, all cases, and output. Java and C++ compilers were unavailable in that workspace, so their team-confirmed passes are retained while timing and toolchain fields remain blank.

Cross-language findings

Python is the most concise implementation for both problems. Java and C++ require more explicit type, class, and runner structure. JavaScript is compact but relies on runtime validation. Observer has the same recorded cyclomatic complexity in all four languages; Strategy uses more explicit branches outside Python.

Evidence

Part3/evidence contains per-language logs, raw timings, summaries, metric rules, human verification, and the detailed comparison. Part3/test_cases contains the shared inputs and expected outputs.

Conclusion

The three-part study is complete. Part 2 is supported by 13 passing tests and a successful MILP smoke test. Part 3 shows that Strategy and Observer keep the same intent across four languages while implementation size and ceremony vary.

The team must now add its source files to the intentionally empty Part1/code, Part2/code, and Part3/code directories. Optional screenshots and Java/C++ raw timing evidence may also be added before submission.